Manual (de)initialization

A Rust and C++ showcase

RUSTIKON
Powered by SoftwareMill

exein

Aleksander Krauze
github.com/aleksanderkrauze
aleksander.krauze@zoho.com
exein.io

```
1 trait Exein: Rust + Security {}
```

About

Boring code

RUSTIKON
Powered by SoftwareMill

exe1n

```rust
pub struct Foo {
    name: String,
    data: Vec<i32>,
    len: usize,
}

impl Foo {
    pub fn new(name: &str) -> Self {
        let name = name.to_owned();
        let len = name.len();

        Self {
            name,
            data: Vec::new(),
            len,
        }
    }
}

fn main() {
    let foo = Foo::new("Rustikon");
}
```

```cpp
#include <string>
#include <vector>

class Foo {
  std::string m_name;
  std::vector<int> m_data;
  std::size_t m_len;

public:
  explicit Foo(const char *name)
      : m_name{name}
      , m_data{}
      , m_len{this->m_name.length()}
    {}
};

int main() {
  Foo foo = Foo{"Rustikon"};

  return 0;
}
```

Manual memory initialization in Rust

`std::mem::MaybeUninit`

```rust
use std::mem::MaybeUninit;
use std::ptr;

pub struct Foo {
    name: String,
    data: Vec<i32>,
    len: usize,
}

impl Foo {
    pub fn new_in<'storage>(
      this: &mut MaybeUninit<Self>,
      name: &str
    ) -> &'storage mut Self {
      let this = this.as_mut_ptr();

      let name = name.to_owned();
      let data = Vec::new();

      unsafe {
        ptr::write(&raw mut (*this).name, name);
        ptr::write(&raw mut (*this).data, data);

        let len = unsafe {
            let name = &*(&raw const (*this).name);
            name.len()
        };
        ptr::write(&raw mut (*this).len, len);
      }

      unsafe { &mut *this }
    }
}

fn use_foo(_foo: &Foo) {}

fn main() {
    let mut storage = MaybeUninit::uninit();

    let foo = Foo::new_in(&mut storage, "Rustikon");
    use_foo(&*foo);

    unsafe {
        storage.assume_init_drop();
    }
}
```

Manual memory initialization in C++

```cpp
#include <string>
#include <vector>

class Foo {
  std::string m_name;
  std::vector<int> m_data;
  std::size_t m_len;

public:
  explicit Foo(const char *name)
      : m_name{name},
      m_data{},
      m_len{this->m_name.length()}
    {}
};

void use_foo(const Foo &foo) {}
```

```cpp

int main() {
  alignas(Foo) std::byte buf[sizeof(Foo)];

  Foo *foo = new (buf) Foo{"Rustikon"};
  use_foo(*foo);
  foo->~Foo();

  return 0;
}
```

Destructors, Drop, and Drop-glue

Normal memory deinitialization in C++

```cpp
#include <print>
#include <string>

class Logger {
  std::string m_message;

public:
  Logger(const char *message) : m_message{message} {}
  ~Logger() { std::println("{}", m_message); }
};

class Foo {
  Logger a;
  Logger b;
  Logger c;

public:
  Foo() : a{"a"}, b{"b"}, c{"c"} {}
  ~Foo() { std::println("Foo"); }
};

int main() {
  Foo foo = Foo{};

  return 0;
}
```

```
Output:
Foo
c
b
a
```

Normal memory deinitialization in Rust

```rust
pub struct Logger(String);

impl Drop for Logger {
    fn drop(&mut self) {
        println!("{}", self.0);
    }
}

pub struct Foo {
    a: Logger,
    b: Logger,
    c: Logger,
}

impl Drop for Foo {
    fn drop(&mut self) {
        println!("Foo");
    }
}

impl Foo {
    pub fn new() -> Self {
        Self {
            a: Logger("a".to_owned()),
            b: Logger("b".to_owned()),
            c: Logger("c".to_owned()),
        }
    }
}

fn main() {
    let foo = Foo::new();
}
```

```
Output:
Foo
a
b
c
```

Controlling drop order

1. Disable drop-glue
2. Run destructors manually

`std::mem::ManuallyDrop`

```rust
use std::mem::ManuallyDrop;

pub struct Logger(String);

impl Drop for Logger {
    fn drop(&mut self) {
        println!("{}", self.0);
    }
}

struct Foo {
    a: ManuallyDrop<Logger>,
    b: ManuallyDrop<Logger>,
    c: ManuallyDrop<Logger>,
}

impl Drop for Foo {
    fn drop(&mut self) {
        unsafe {
            ManuallyDrop::drop(&mut self.a);
            ManuallyDrop::drop(&mut self.c);
            ManuallyDrop::drop(&mut self.b);
        }
    }
}

```

```rust
impl Foo {
    fn new() -> Self {
        Self {
            a: ManuallyDrop::new(Logger("a".to_owned())),
            b: ManuallyDrop::new(Logger("b".to_owned())),
            c: ManuallyDrop::new(Logger("c".to_owned())),
        }
    }
}

fn main() {
    let foo = Foo::new();
}
```

```
Output:
a
c
b
```

C++

With runtime overhead:

- std::optional
- std::unique_ptr

`union`

```cpp
#include <print>
#include <string>

class Logger {
  std::string m_message;

public:
  Logger(const char *message) : m_message{message} {}
  ~Logger() { std::println("{}", m_message); }
};

class Foo {
  union {
    Logger a;
  };
  union {
    Logger b;
  };
  union {
    Logger c;
  };

public:
  Foo() : a{"a"}, b{"b"}, c{"c"} {}
  ~Foo() {
    a.~Logger();
    c.~Logger();
    b.~Logger();
  }
};

int main() {
  Foo foo = Foo{};

  return 0;
}
```

```
Output:
a
c
b
```

Summary

	Rust	C++
initialization order	unspecified	In the order of declaration
in-place initialization	Manual. Open design space	Yes. Placement new
destruction order	In the order of declaration	In the *reverse* order of declaration
controlling destruction order	`ManuallyDrop`	`union`

RUSTIKON
Powered by SoftwareMill

exein

Public repository with slides